

59

Accès aux Bases de Données (PDO / MySQLi)

59

Méthodes de connexion

60

PHP communique avec MySQL via deux méthodes :

- PDO (recommandé) ✓
- MySQLi
- **PDO (PHP Data Objects)** : C'est une interface moderne, sécurisée et orientée objet qui fonctionne avec plusieurs types de bases de données (MySQL, PostgreSQL, etc.).
- **PDO** est une **extension de PHP** qui permet de se connecter et de manipuler des bases de données.
- **PDO** = outil pour communiquer entre PHP et la base de données

60

Connexion à la base (avec PDO)

61

- Il faut quatre informations : l'hôte, le nom de la base, l'utilisateur et le mot de passe.

```
<?php
$pdo = new PDO("mysql:host=localhost;dbname=test", "root", "");
?>
```

```
<?php
$host = 'localhost';
$db   = 'ma_base_l2';
$user = 'root';
$pass = ''; // Vide par défaut sous WAMP/XAMPP
$dsn  = "mysql:host=$host;dbname=$db;charset=utf8mb4";

try {
    $pdo = new PDO($dsn, $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connexion réussie !";
} catch (PDOException $e) {
    die("Erreur de connexion : " . $e->getMessage());
}
?>
```

61

Connexion à la base (avec MySQLi)

62

- Il faut quatre informations : l'hôte, l'utilisateur, le mot de passe et le nom de la base.

```
<?php
$conn = new mysqli("localhost", "root", "", "test");

if ($conn->connect_error) {
    die("Erreur de connexion");
}
?>
```

62

Pourquoi utiliser PDO ?

63

- Compatible avec plusieurs bases :
 - ▣ MySQL
 - ▣ PostgreSQL
 - ▣ SQLite
- Code plus **portable**
- Plus **sécurisé**
- Support des **requêtes préparées** (anti-injection SQL)

Critère	PDO	MySQLi
Multi-base	✓	✗ (MySQL seulement)
Sécurité	✓	✓
Requêtes préparées	✓	✓
Facilité	✓	✓

63

Le CRUD (Create, Read, Update, Delete)

64

- La méthode **exec()** exécute une requête SQL et retourne **uniquement le nombre de lignes affectées** par cette opération (combien de lignes ont été supprimées, modifiées ou ajoutées).
- Elle ne retourne JAMAIS de jeu de résultats (pas de données à afficher).
- On l'utilise exclusivement pour les requêtes de type **Action** :
 - INSERT (Ajouter), UPDATE (Modifier), DELETE (Supprimer)
- La méthode **query()** est l'une des fonctions les plus utilisées de l'objet PDO pour interagir avec une base de données. Elle est spécifiquement conçue pour **récupérer des données**.
- Elle permet d'exécuter une requête SQL (généralement un SELECT) et retourne immédiatement un **jeu de résultats** sous forme d'objet PDOStatement.

64

Le CRUD (Create, Read, Update, Delete)

65

- C'est le cycle de vie des données.

A. CREATE (Insertion)

```
<?php
$sql = "INSERT INTO etudiants (nom, age) VALUES ('Ali', 20)";
$pdo->exec($sql);
?>
```

B. READ (Lecture)

```
<?php
$sql = "SELECT * FROM etudiants";
$result = $pdo->query($sql);

foreach ($result as $row) {
    echo $row["nom"] . "<br>";
}
?>
```

65

Le CRUD (Create, Read, Update, Delete)

66

C. UPDATE (Mise à jour)

```
<?php
$sql = "UPDATE etudiants SET age = 22 WHERE nom = 'Ali'";
$pdo->exec($sql);
?>
```

D. DELETE (Suppression)

```
<?php
$sql = "DELETE FROM etudiants WHERE nom = 'Ali'";
$pdo->exec($sql);
?>
```

66

Le CRUD (Create, Read, Update, Delete)

67

❏ Sécurité

- **Requête préparée (PDO)** : protège la base contre injection SQL
- C'est la méthode la plus efficace. Au lieu d'envoyer la requête avec les données mélangées, on envoie d'abord la "forme" de la requête, puis les données séparément.

```
<?php
$stmt = $pdo->prepare("INSERT INTO etudiants (nom, age) VALUES (?, ?)");
$stmt->execute(["Ali", 20]);
?>
```

67

Le CRUD (Create, Read, Update, Delete)

68

A. **CREATE (Insertion avec préparation)**

```
<?php
$sql = "INSERT INTO etudiants (nom, prenom, moyenne) VALUES (?, ?, ?)";
$stmt = $pdo->prepare($sql);
$stmt->execute(["Ben Foulen", "Foulen", 15.5]);
echo "Étudiant ajouté !";
?>
```

B. **READ (Lecture)**

```
<?php
$query = $pdo->query("SELECT * FROM etudiants");
while ($row = $query->fetch()) {
    echo $row['nom'] . " " . $row['prenom'] . "<br>";
}
?>
```

68

Le CRUD (Create, Read, Update, Delete)

69

c. UPDATE (Mise à jour)

```
<?php
$sql = "UPDATE etudiants SET moyenne = :nouvelle_moyenne WHERE nom = :nom";
$stmt = $pdo->prepare($sql);
$stmt->execute(['nouvelle_moyenne' => 16.0, 'nom' => 'Ben Foulen']);
?>
```

d. DELETE (Suppression)

```
<?php
$sql = "DELETE FROM etudiants WHERE id = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([1]); // Supprime l'étudiant avec l'ID 1
?>
```

69

Récap.

70

1. Architecture & Fonctionnement

- ❑ **Côté Serveur** : Le PHP est exécuté sur le serveur (Apache). Le client (navigateur) ne reçoit que du HTML.
- ❑ **Environnement** : Nécessite une pile **WAMP, MAMP ou XAMPP**.
- ❑ **Syntaxe** : Tout code PHP est délimité par `<?php ... ?>`. Chaque instruction se termine par un **point-virgule ;**.

2. Variables et Types

- ❑ Déclaration avec le symbole **\$** (ex: `$moyenne = 15.5;`).
- ❑ **Concaténation** : On utilise le point `.` (ex: `"Bonjour " . $nom`).
- ❑ **Tableaux Associatifs** : Clés textuelles pour organiser les données.

```
$etudiant = ["nom" => "Ben Foulen", "classe" => "L2"];
```

70

Récap.

71

3. Communication Client-Serveur

- **\$_GET** : Données dans l'URL (visible, limité, pour la consultation /recherche).
- **\$_POST** : Données dans le corps de la requête (invisible, sécurisé, pour l'envoi de formulaires).
- **Sécurité** : Toujours utiliser htmlspecialchars() pour éviter les failles XSS.

4. Persistance et État

- **Sessions** : Stockées sur le serveur.
Nécessitent session_start() au début du fichier.
Utiles pour l'authentification.
- **Cookies** : Stockés chez le client (navigateur).
Utiles pour les préférences (langue, thème).

71

Récap.

72

5. Bases de Données (PDO)

- **Connexion** : Utilisation d'un objet new PDO().
- **Sécurité SQL** : Ne jamais injecter de variables directement dans une requête.
 - Utiliser les **requêtes préparées** (prepare et execute) pour bloquer les injections SQL.

72

Manipulation des tableaux

73

□ **Fonctions de Tri (Sorting) :** Ces fonctions modifient directement le tableau original (passage par référence interne).

Fonction	Rôle	Exemple	Résultat
<code>sort()</code> / <code>rsort()</code>	Tri croissant / décroissant simple.	<code>\$t = [10, 5, 20]; sort(\$t);</code>	<code>[5, 10, 20]</code>
<code>asort()</code> / <code>arsort()</code>	Tri par valeur en gardant le lien Clé => Valeur .	<code>\$n = ["Math"=>15, "PHP"=>10]; asort(\$n);</code>	<code>["PHP"=>10, "Math"=>15]</code>
<code>ksort()</code> / <code>krsort()</code>	Tri par Clé (alphabétique ou numérique).	<code>\$e = ["Z"=>1, "A"=>2]; ksort(\$e);</code>	<code>["A"=>2, "Z"=>1]</code>

73

Manipulation des tableaux

74

□ **Fonctions de Recherche et d'Extraction :** Ces fonctions retournent une nouvelle valeur ou un nouveau tableau sans modifier l'original.

Fonction	Rôle	Exemple	Résultat
<code>array_keys()</code>	Récupère toutes les clés du tableau.	<code>\$etudiant = ["nom"=>"Ben Foulen", "L2"=>"Info"]; array_keys(\$etudiant);</code>	<code>["nom", "L2"]</code>
<code>array_values()</code>	Récupère toutes les valeurs du tableau.	<code>array_values(\$etudiant);</code>	<code>["Ben Foulen", "Info"]</code>
<code>array_search()</code>	Cherche une valeur et retourne sa clé .	<code>\$v = ["Rouge", "Vert"]; array_search("Vert", \$v);</code>	<code>1 (l'indice)</code>

74

Manipulation des tableaux

75

□ Différence entre **sort()** et **asort()**

```
<?php
$notes = ["Algo" => 18, "Système" => 10, "Réseau" => 14];

// Cas A : On utilise sort (Erreur classique)
$test1 = $notes;
sort($test1);
// Résultat : Les noms des matières disparaissent, remplacés par 0, 1, 2.

// Cas B : On utilise asort (La bonne méthode)
$test2 = $notes;
asort($test2);
// Résultat : On garde "Système" => 10, "Réseau" => 14, etc.
?>
```

75

Gestion des zones de texte (Textarea)

76

```
<form action="traitement.php" method="POST">
  <p><b>Votre Message :</b><br />
  <textarea name="message" rows="5" cols="30">Tapez ici...</textarea></p>
  <p><input type="submit" value="Envoyer"></p>
</form>
```

```
<?php
if (isset($_POST['message'])) {
  $msg = $_POST['message'];
  // nl2br transforme les retours à la ligne (\n) en balises HTML <br />
  echo "Message reçu : <br />" . nl2br(htmlspecialchars($msg));
}
?>
```

Votre Message :

Bonjour,
Comment allez vous ?
How are You ?

Message reçu :
Bonjour,
Comment allez vous ?

How are You ?

76

Gestion des boutons d'envoi multiples

77

```
<form action="action.php" method="POST">
  <p>Utilisateur : <input type="text" name="user"></p>
  <p>
    <input type="submit" name="action" value="Ajouter">
    <input type="submit" name="action" value="Supprimer">
  </p>
</form>
```

Utilisateur :

Ajouter Supprimer

```
<?php
if (isset($_POST['action'])) {
    $choix = $_POST['action'];
    $nom = $_POST['user'];

    if ($choix == "Ajouter") {
        echo "L'utilisateur $nom a été ajouté.";
    } elseif ($choix == "Supprimer") {
        echo "L'utilisateur $nom a été supprimé.";
    }
}
?>
```

Utilisateur :

Ajouter Supprimer

L'utilisateur Foulen a été ajouté.

Utilisateur :

Ajouter Supprimer

L'utilisateur Foulen a été supprimé.

77

Gestion du sélecteur de date (Date Picker)

78

```
<form action="traitement_date.php" method="POST">
  <p>Date de naissance :
  <input type="date" name="date_naiss"></p>
  <input type="submit" value="Valider">
</form>
```

Date de naissance : 

Valider

```
<?php
if (isset($_POST['date_naiss'])) {
    $date = $_POST['date_naiss'];
    // On peut utiliser strtotime pour transformer la date en format français
    echo "Date reçue : " . $date . "<br>";
    echo "Format français : " . date("d/m/Y", strtotime($date));
}
?>
```

Date de naissance : 

Valider

Date de naissance : 

Valider

Date reçue : 1995-02-15
Format français : 15/02/1995

78

Gestion des boutons radios (choix unique)

79

```
<p>Quel est votre niveau d'études ?</p>
<form action="reponse.php" method="POST">
  <input type="radio" name="niveau" value="L1"> Licence 1 <br>
  <input type="radio" name="niveau" value="L2" checked> Licence 2 <br>
  <input type="radio" name="niveau" value="L3"> Licence 3 <br>
  <input type="submit" value="Envoyer">
</form>
```

```
<?php
if (isset($_POST['niveau'])) {
    $niveau = $_POST['niveau'];
    echo "Votre niveau est : " . $niveau;
}
else {
    echo "Veuillez sélectionner un niveau d'études.";
}
?>
```

Quel est votre niveau d'études ?

- ☐ Licence 1
☒ Licence 2
☐ Licence 3

Votre niveau est : L2

79

Gestion des Checkbox (Choix Multiples)

80

```
<p>Quelles sont vos compétences ?</p>
<form action="competences.php" method="POST">
  <input type="checkbox" name="skills[]" value="HTML"> HTML <br>
  <input type="checkbox" name="skills[]" value="PHP"> PHP <br>
  <input type="checkbox" name="skills[]" value="Python"> Python <br>
  <input type="submit" value="Enregistrer">
</form>
```

```
<?php
// On vérifie si au moins une case a été cochée
if (isset($_POST['skills'])) {
    $mes_compétences = $_POST['skills']; // C'est un tableau (Array)

    echo "Vos compétences sont : <ul>";
    foreach ($mes_compétences as $s) {
        echo "<li>" . htmlspecialchars($s) . "</li>";
    }
    echo "</ul>";
} else {
    echo "Aucune compétence sélectionnée.";
}
?>
```

Quelles sont vos compétences ?

- ☐ HTML
☐ PHP
☐ Python

Aucune compétence sélectionnée.

- ☒ HTML
☒ PHP
☒ Python

Vos compétences sont :

- HTML
- PHP
- Python

80

81

QCM

81

QCM

82

1. **Lesquelles de ces syntaxes sont correctes pour déclarer une variable en PHP ?**

- A) `var $nom = "Foulen";`
- B) `$nom = "Foulen";`
- C) `string $nom = "Foulen";`
- D) `$_nom = "Foulen";`

2. **Soit le tableau `$tab = ["A", "B", "C"]`. Quelle instruction affiche "B" ?**

- A) `echo $tab[0];`
- B) `echo $tab[1];`
- C) `echo $tab["B"];`

82

QCM

83

3. Concernant les méthodes GET et POST, quelles affirmations sont vraies ?

- A) GET affiche les données dans l'URL.
- B) POST est limité en taille de données.
- C) POST est préférable pour l'envoi de mots de passe.
- D) On récupère les données GET via la superglobale \$_GET.

4. À quoi sert la fonction htmlspecialchars() ?

- A) À crypter les mots de passe.
- B) À empêcher les failles XSS en neutralisant les balises HTML.
- C) À connecter PHP à la base de données.

83

QCM

84

5. Où doit être placée la fonction session_start() ?

- A) À la toute fin du script.
- B) Juste avant de fermer la balise ?>.
- C) Au tout début du fichier, avant tout code HTML.

6. Pourquoi utiliser des "requêtes préparées" avec PDO ?

- A) Pour rendre le code plus court.
- B) Pour se protéger contre les injections SQL.
- C) Pour améliorer les performances lors d'exécutions répétées.

7. Quel bloc permet de capturer une erreur de connexion à la BD ?

- A) if / else
- B) try / catch
- C) foreach / as

84

QCM

85

- Quel sera l'affichage du script suivant ?

```
$notes = ["Alice" => 15, "Bob" => 10];  
foreach ($notes as $n) {  
    $n = $n + 2;  
}  
echo $notes["Alice"];
```

- A) 17
- B) 15
- C) Une erreur PHP s'affiche.

85